Daniel Pedder
Daniel.Pedder@tha.de

Mahalakshmi Krishnan
Mahalakshmi.Krishnan@tha.de

Thomas Homm
Thomas.Homm@tha.de

***Abstract*—This project covers the design and hardware implementation of a 50 MHz baseband Direct-Sequence Spread Spectrum transmitter receiver, as well as a AWGN channel emulator. Each of these components is implemented using SpinalHDL and tested on an Altera Cyclone II FPGA equipped with external 14-Bit DAC/ADCs. To optimize design parameters, Monte-Carlo-Simulations are employed.**

I. Introduction

Direct-Sequence spread spectrum (DSSS) modulation is the basis of many modern wireless systems, a famously known example is modern GPS. The idea behind DSSS is simple, a transmitter spreads a single data bit into many chips. Chips are created using a sequence generated from a Linear Feedback Shift Register (LFSR), which is based on a set polynominal. A receiver that knows the same chip sequence polynominal can despread the signal and recover the original data, while other receivers that do not know the exact code sequence only sees noise and can't descramble the incoming chips. This property gives DSSS systems a decent resistance to interference and also allows different users to share the same frequency band simultaneously [1].

In this Master Project, the goal of the project is building a working DSSS transmitter and receiver in hardware on a Cyclone II FPGA. A receiver does not know when a transmission starts or what phase of the spreading code it will see first, and must search for the correct code phase before it can track it. Once it found the correct code, it considers it self synchronized. Small clock differences between transmitter and receiver mean the receiver must constantly adjust its own code phase to stay locked onto the transmitter. These problems become harder under the constraints of a FPGA system running from a 50 MHz clock and no floating-point hardware.

This project addresses these problems by designing and implementing a complete DSSS communication system in hardware. The project splits the system across three separate FPGA boards. A transmitter which spreads and outputs the individual chips over an ADC add-on board, a channel which injects attenuation and noise, and a receiver that must acquire and track the spreading code.

The whole project is implemented using the SpinalHDL language, a Scala based hardware description language, which compiles into Verilog or VHDL. This project also considers the impacts of SpinalHDL on the whole design and implementation process. It serves as a test of how well SpinalHDLs abstractions hold up on a decently complex design. The receiver is the central focus of this work, since it has to solve both the code acquisition and code tracking problems.

This projects documentation begins with the theoretical background needed to follow the rest of the thesis, covering cross-correlation, the behaviour of signals in noise, and the principles of Direct-Sequence Spread Spectrum. This is followed by a description of the system design and FPGA implementation, starting with a short introduction to SpinalHDL itself, before covering the transmitter, the emulated channel, and the receiver's acquisition and tracking logic in detail. The final chapters present the simualtion of the system, the results and conclude the projects documentation.

II. Theoretical Background

This chapter discusses the theoretical background before moving to the actual implementation of the design on hardware, beginning with Cross Correlation.

A. Cross Correlation

Cross-correlation quantifies the similarity between two signals as a function of a relative time shift τ. For two real, discrete-time sequences $x[n]$ and $y[n]$, the cross-correlation is defined as

$$R_{\{xy\}}[k] = \sum_{\{n=-\infty\}}^{\{\infty\}} x[n]\, y[n+k] \tag{1}$$

When $x = y$, this reduces to the autocorrelation $R_{\{xx\}}[k]$, which measures how similar a signal is to a time-shifted copy of itself. Autocorrelation is central to DSSS because the spreading sequence's own correlation properties determine how sharply a receiver can localize the correct code phase.

a) *Properties relevant to spreading codes:*

For a pseudorandom noise (PN) sequence $c[n] \in \{+1, -1\}$ of period N, Autocorrelation matters for an ideal spreading code has a sharply peaked autocorrelation function, a single high value at zero lag ($k = 0$) and low, ideally near-zero, values at all other lags:

$$R_{\{cc\}}[k] = \begin{cases} N & \text{if } k \equiv 0 \bmod N \\ -1 \ \text{(or small)} & \text{otherwise} \end{cases} \tag{2}$$

[2]

For example, the GPS C/A code has an off-peak autocorrelation value of exactly $-1/1023$, normalized to a unity peak [2]. This is what allows a receiver to identify the correct code phase: any misalignment collapses the correlation output toward zero, while perfect alignment produces a strong peak. Maximal-length LFSR sequences (m-sequences) of length $N = 2^n - 1$ achieve exactly this two-valued autocorrelation, with off-peak value -1. This project relies on this property directly for its 10-bit LFSR chip generator, which produces the same 1023-chip code length used by GPS.

b) *The sliding correlator:*

Practically, a receiver does not know the incoming code phase in advance. It computes $R_{\{rc\}}[k]$ between the received chip stream $r[n]$ and a locally generated, phase-shifted

replica of the code $c[n+k]$ for a range of trial shifts k. This is the sliding correlator, and the shift k that maximizes $|R_{\{rc\}}[k]|$ is the acquired code phase [2]. This operation is the theoretical basis for the acquisition stage implemented in the receiver's correlator logic, discussed in the system design chapter.

B. Signals and Noise

a) *Channel model:*

The channel between transmitter and receiver is commonly modeled as an Additive White Gaussian Noise (AWGN) channel:

$$r(t) = s(t) + n(t) \tag{3}$$

where $s(t)$ is the transmitted (spread) signal and $n(t)$ is zero-mean Gaussian noise with power spectral density $N_0/2$ (two-sided). "White" means the noise power is uniformly distributed across frequency, so it corrupts every chip independently. This is the idealized model this project's channel board targets As described in Channel Emulation, the FPGA has no floating-point hardware and so instead of true Gaussian noise, the channel sums the outputs of five independent LFSRs and relies on the Central Limit Theorem to approximate a Gaussian-like distribution.

The Central Limit Theorem states that the sum of enough independent random variables tends toward a Gaussian distribution, regardless of how the individual variables themselves are distributed. Each LFSRs own output bit is nowhere near Gaussian on its own, but adding together several roughly-uniform sources pulls the combined result toward a proper bell curve. This is exactly the mechanism the channel board exploits to approximate AWGN.

b) *SNR and E_b/N_0:*

Two related figures of are used throughout this thesis, SNR, defined as$P_{\text{signal}}/P_{\text{noise}}$, ratio of signal to noise power (often per chip in DSSS). The other is E_b/N_0, defined as Energy per information bit relative to noise power spectral density, the standard measure for comparing digital modulation schemes independent of bandwidth.

Because DSSS trades bandwidth for robustness, the chip-level SNR seen by the receiver can be considerably worse than the bit-level SNR recovered after despreading. This gap is exactly the processing gain discussed below. The project's Python BER-vs-SNR sweep (ber_vs_snr.py) reports results in chip-level SNR, since that's the quantity the channel board's noise and attenuation settings directly control. [3]

c) *Matched filtering and the correlation receiver:*

For a known pulse shape in AWGN, the linear filter that maximizes the output SNR at the sampling instant is the matched filter, whose impulse response is the time-reversed, conjugated transmit pulse:

$$h(t) = s(T_b - t) \tag{4}$$

[3]

For rectangular chip/bit pulses (as used in this project's PAM-style baseband signalling), the matched filter is equivalent to an integrate-and-dump correlator, which is, multiplying the received signal by a local replica of the expected waveform and integrating over the symbol period. This is precisely what the DSSS despreading correlator does, it is a matched filter matched to the known chip sequence, which is why coherent code alignment is essential for correct data recovery.

d) *Bit error rate:*

For binary antipodal signalling (BPSK-equivalent, as used for the chip values $\in \{+1, -1\}$) in AWGN, the theoretical bit error probability after matched filtering is

$$P_b = Q!\left(\sqrt{2E_b/N_0}\right) \tag{5}$$

[3]

where $Q(x) = \frac{1}{\sqrt{2\pi}} \int_x^\infty e^{\{-u^2/2\}} du$ is the Gaussian tail function [3]. This closed-form expression is the benchmark against which the project's simulated BER-vs-SNR curves (produced by the Python modeling suite) are compared, and against which the measured coding/processing gain of the spreading system is quantified.

e) *The Shannon-Hartley theorem and spreading:*

The Shannon-Hartley theorem [4]

$$C = W \log\left(1 + \frac{P}{N_0 W}\right) \tag{6}$$

states that for a given power P and noise spectral density N_0, the channel capacity C asymptotically approaches a certain limit with increasing channel bandwidth W, as shown in Fig. 1. From this follows that a signal can be "spread" across the spectrum arbitrarily, without increasing transmit power, even though the total noise power in the channel increases.

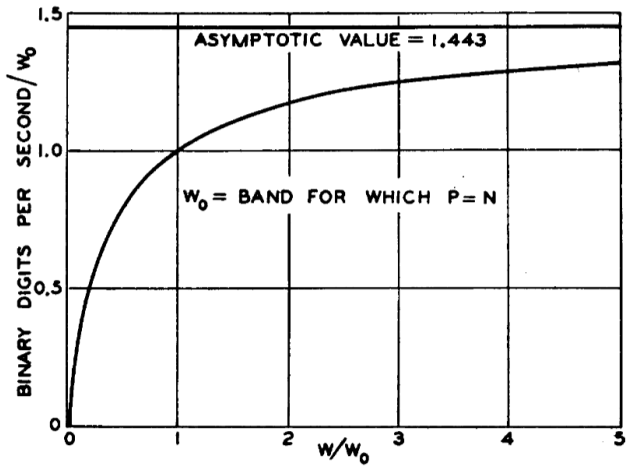

Fig. 1. Channel capacity in relationship with the noise-equivalent bandwidth W_0 and actual bandwidth W, from [4].

To interfere with such a signal using a broadband jammer, the required power to reduce the received signal quality significantly can be arbitrarily large with increasing spreading factor, also referred to as processing gain

$$G = \frac{W_s}{W} \tag{7}$$

where W is the signal bandwidth before, and W_s the signal bandwidth after the spreading operation.

C. Direct-Sequence Spread Spectrum

a) *Spreading operation:*

A DSSS transmitter multiplies each data bit $b \in \{+1, -1\}$, held constant for one bit period T_b, by a PN chip sequence $c[n] \in \{+1, -1\}$ running at a much higher rate:

$$s[n] = b \cdot c[n], \quad n = 0, ..., N_c - 1 \tag{8}$$

[5]

where N_c chips are transmitted per data bit. In hardware this multiplication is implemented as an XOR between the data bit and the LFSR-generated chip, which is logically equivalent to ± 1 multiplication on bipolar values, which is the basis of the Transmitter module in this project. In this project, $N_c = 1023$, matching the GPS C/A code length discussed below.

b) *Processing gain:*

The ratio of chip rate to bit rate defines the processing gain:

$$G_p = \frac{R_c}{R_b} = \frac{T_b}{T_c} = N_c \tag{9}$$

[5]

expressed in decibels as $G_p[\text{dB}] = 10 \log_{10}(N_c)$. For GPS, this definition gives a C/A-code processing gain of approximately 43 dB (chip rate 1.023 Mchip/s against a 50 bit/s navigation data rate), which combines with the required post-correlation E_b/N_0 and an implementation loss term to give the system's jamming margin,

$$M_j[\text{dB}] = G_p[\text{dB}] - (E_b/N_0)_{\text{req}}[\text{dB}] - L_{\text{impl}}[\text{dB}] \tag{10}$$

[2], [6]

Processing gain is the theoretical source of the despreading correlator's noise-averaging benefit: narrowband interference and noise, uncorrelated with the code, are spread out (and thus attenuated) by the despreading multiplication, while the wanted signal, correlated with the local code replica, is coherently reconstructed. This matches the project's empirical finding that increasing chip rate improves the effective coding gain (9 dB coding gain observed at chip_rate $= 8$, sufficient to drive BER to 0% at 3 dB SNR); the choice of chip/sampling rate relative to the code rate is well known to affect DLL tracking performance in exactly this way [7], [8].

c) *Despreading and code synchronization:*

At the receiver, despreading is the inverse operation: the incoming chip stream is correlated against a locally generated, correctly phase-aligned replica of the same PN code and integrated over one bit period:

$$\hat{b} = \text{sign}\left(\sum_{\{n=0\}}^{\{N_c-1\}} r[n]\, c[n] \right) \tag{11}$$

[5]

Correct operation depends entirely on the local code being phase-aligned with the incoming code, this is the code synchronization problem, which splits into two sub-problems:

- **Acquisition**: coarse alignment, found via the sliding correlator search described above, declaring "synchronized" once the correlation peak exceeds a detection threshold.
- **Tracking**: fine, continuous alignment maintained after acquisition. Since transmitter and receiver clocks are never perfectly matched, small frequency offsets (measured in ppm) cause the code phases to slowly drift apart over time. The correlation peak would walk away from the sample instant if left uncorrected. A Delay-Locked Loop (DLL) correlates the incoming signal against an **early** ($c[n + \delta]$) and a **late** ($c[n - \delta]$) replica of the local code, offset by a fraction of a chip δ, and forms the normalized early-late discriminator

$$D = \frac{|R_E| - |R_L|}{|R_E| + |R_L|} \tag{12}$$

[2], [9], [10]

whose sign and magnitude drive the local code clock back toward alignment, keeping the correlation peak centered, the same early-late DLL architecture used for code tracking in GPS receivers. This directly motivates the DLL-based tracking logic implemented for the receiver in this project, needed once uncorrected offsets in the 5–10 ppm range were shown to cause synchronization failure.

d) *LFSR-generated PN sequences:*

The chip sequences are generated by Linear Feedback Shift Registers implementing primitive polynomials over $\text{GF}(2)$. An n-bit LFSR with a primitive feedback polynomial produces a maximal-length sequence (m-sequence) with period

$$N = 2^n - 1 \tag{13}$$

[11]

M-sequences have three properties (the "balance", "run", and "correlation" properties) that make them suitable spreading codes [11]:

1) **Balance**: in one period, the number of 1s exceeds the number of 0s by exactly one.
2) **Run distribution**: runs of consecutive equal chips occur with geometrically decreasing frequency, mimicking a random binary sequence.
3) **Two-valued autocorrelation**: as given above, $R_{\{cc\}}[k] = N$ at $k = 0$ and -1 elsewhere — the property directly exploited for acquisition.

This project uses a 10-bit LFSR, yielding $N = 1023$ chips per period, the basis of the PRNS generator used both in the FPGA transmitter/receiver and in the Python reference model for offline BER and tracking analysis. This is deliberately the same code length used by the GPS C/A code, which is also generated from 10-bit shift registers at a chip rate of 1.023 Mchip/s [2].

e) *Multiple access and the near–far problem:*

Because distinct users' codes have low mutual cross-correlation, several DSSS transmissions can share the same frequency band simultaneously (Code-Division Multiple Access) with each receiver's correlator suppressing the others as pseudo-noise [11]. This robustness is not unconditional, however: if an interfering transmitter's received power is much larger than the desired signal's, residual cross-correlation sidelobes can still swamp the wanted correlation peak, which is why power control or attenuation matching matters

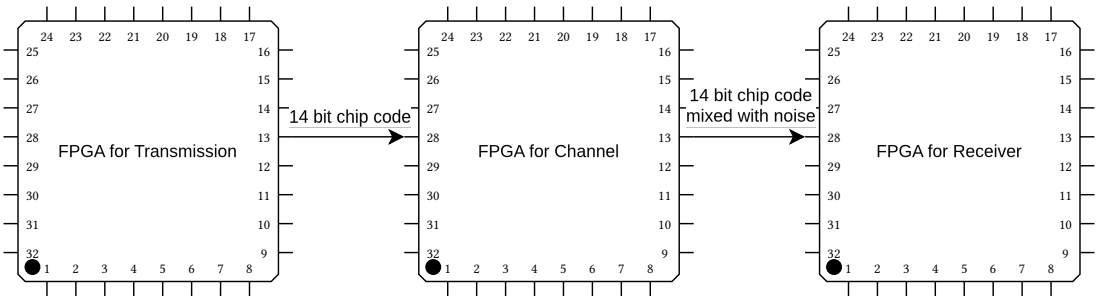

Fig. 2. System Overview

in the channel emulation stage of this project's three-board test setup.

III. System Design and Implementation

The Cyclone II FPGA development boards provide a 50 MHz clock, which is selected as both the system clock rate and the chip rate of the spreading sequence. The decision to use the maximum available clock was made during early project meetings to be as 'fast as possible'.
The system itself has three main components, the transmitter generates and sends out the spread signal, the channel simulates noise and tries to simulate a 'real world scenario', and the receiver acquires and tracks the spreading code to try to rebuild the original data from the transmitter. This chapter describes the main components implementation and the whole system overview in detail.
The chapter begins with a small overview, then introduces SpinalHDL as the implementation language, next the generation of the spreading sequence generation using LFSRs, followed by the transmitter, receiver, and channel designs, and closes with the top-level FPGA integration and a discussion of the current design's known limitations.

A. System Overview

The 3 main components of the system, the transmitter, the channel, and the receiver, also correspond to the three FPGA boards used, each component having its own FPGA board.
The transmitter begins by sending out data. It spreads a single data bit into 1023 chips. For the receiver to be synchronized, the transmitter and receiver have to use the same spreading code, generated from the same LFSR polynomial. The transmitter then sends out these chips through its onboard 14 bit DAC. Between the transmitter and receiver lies the channel. It is made to create artificial noise, which allows simulation of a real world environment. The channel is implemented in a way to make it adjustable on the fly. so users can adjust the attenuation and noise of the signal during system runtime on the FPGA boards by using onboard switches.
The receiver is the most complex part of the system. It receives chips either directly from the transmitter or, usually, from the channel, on its ADC input. It has to try to stay in sync with the transmitter at all times.
A transmission begins with the having to find the transmitters code phase in the first place. This is done by acquisition, where many correlators, each checking a different offset of the LFSR, run in parallel and search for the offset that gives the strongest correlation. Once an offset clearly stands out from the rest, the receiver uses that offset and considers itself synced and begins tracking.
The receiver has to use the same polynomial as the transmitter for its own LFSR, else they could never stay synced. In this projects implementation, three different correlators are run in parallel. One for the expected current bit in the LFSR, one for the previous bit, and one for the next bit in time. We compare each correlators results, and if the earlier or later correlator shows a much better result than the current one, we shift the LFSR forward, or step it back.
This allows us to continuously adjust the LFSR index and correct possible synchronization mistakes.

As shown in Fig. 2, the system chains three FPGAs together sequentially. Because the ADCs use a 14-bit resolution, the transmitter FPGA outputs the 14-bits, representing one chip, to the channel FPGA. This middle channel board then adds the artificial interference, passing the resulting 14-bit chip code mixed with noise into the receiver FPGA for signal acquisition and tracking.

B. SpinalHDL

All designs in this system are written in SpinalHDL, no VHDL or Verilog was writting by hand. SpinalHDL is a hardware description language based on Scala. Hardware components are described as Scala classes, and the SpinalHDL compiler elaborates them into a synthesizable VHDL/Verilog code. Then, Quartus is used for place and route and to programm onto the FPGA.
Because SpinalHDL is embedded into the Scala language, standard Scala constructs like loops, case classes, generics, and collections are all heavily used in the project.
A benefit of using SpinalHDL is that the compiler catches errors before simulation/synthesis runs, which allows for faster debugging.
For verification, the project uses SpinalHDLs own simulation backend, which is used together with Verilator. This allows testbenches to be written directly in SpinalHDL/Scala, rather than writing the testbench in a separate VHDL/Verilog file.

C. Spreading Sequence Generation

The spreading code used throughout the system is generated by an unrolled LFSR class. This LFSR unrolls the feedback logic to produce a full parallel output every cycle. Compared to regular LFSRs, the difference is that we do not just output the resulting bit of the LFSR, but every output

of each tap, of the LFSR. The polynomial itself is passed as an array of tap positions. Each entry in the array represents the exponent of a tap in the polynomial. Because of this, the polynomial of the LFSR is not hardcoded, but adjustable on compile time.
Since we are limited to a 50 MHz clock on both receiver and transmitter, we cannot rely on a faster internal oversampling clock to process multiple LFSR shifts per chip. Instead, the entire state of the UnrollLFSR is exposed at once. This allows to advance the LFSR by standard steps, or to use a skip signal to jump an extra step forward in a single clock cycle, which shifts the LFSR's relative phase without needing a faster tracking clock.

D. Transmitter Implementation

As seen in Fig. 3, the transmitter architecture applies the spreading code to the user input data and send the individual chips out.
The process begins with the unrolled LFSR. When the LFSR completes a full spreading sequence, it asserts a flag signal to indicate the start of a new data symbol cycle. The transmitter uses this flag as an enable signal to latch the next incoming io.data bit into the latchedData register.
This latched data bit is then continuously XORed with the output bit of the LFSR's spreading code, lfsr.io.rnd_o(0).
The transmitter board has an external 14-bit DAC attached. If the spread chip evaluates to a logical 1, it drives the maximum positive value of 14 bits, to the DAC. If false, it drives the minimum negative value instead.

E. Receiver Implementation

The receiver is a more complex implementation, since it has to solve multiple problems on its own. It first has to find the transmitters code phase, and then stay aligned with it. So even if there is clock drift between the two boards, transmitter and receiver should still be aligned.

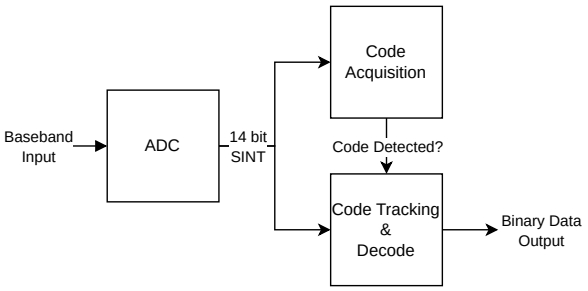

Fig. 4. Overview of the basic building blocks of the receiver. The analog baseband input gets sampled by the ADC. The code acquisition block finds the maximum correlation offset of a local code replica with the signal, at which point the tracking and decode block is enabled.

As shown in Fig. 4, the sampled ADC output first feeds the code acquisition block. Once that block reports the code as detected, the code 'tracking & decode' block is enabled and takes over, decoding the incoming chip stream back into binary data. The following subsections, 'Acquisition' and 'Tracking and Demodulation', describe these in more detail.

a) *Acquisition:* Before the receiver can extract any data, it must find the exact code phase of the incoming signal. This is the reason we use the spreading code.

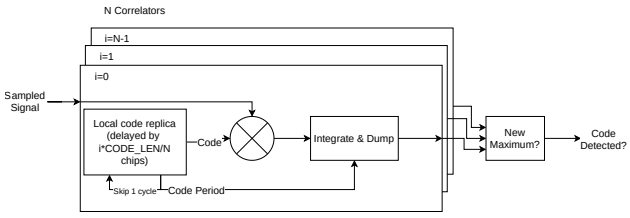

Fig. 5. Simplified diagram of the code acquisition block. N parallel correlators are implemented, each shifted by an equal delay to cover the entire code search space. Whenever a new maximum correlation value is found, the code detected line outputs a positive pulse.

Each of the N correlators in Fig. 5 contains an LFSR configured with the same spreading code polynomial as the transmitter. The correlators are delayed by i*CODE_LEN/

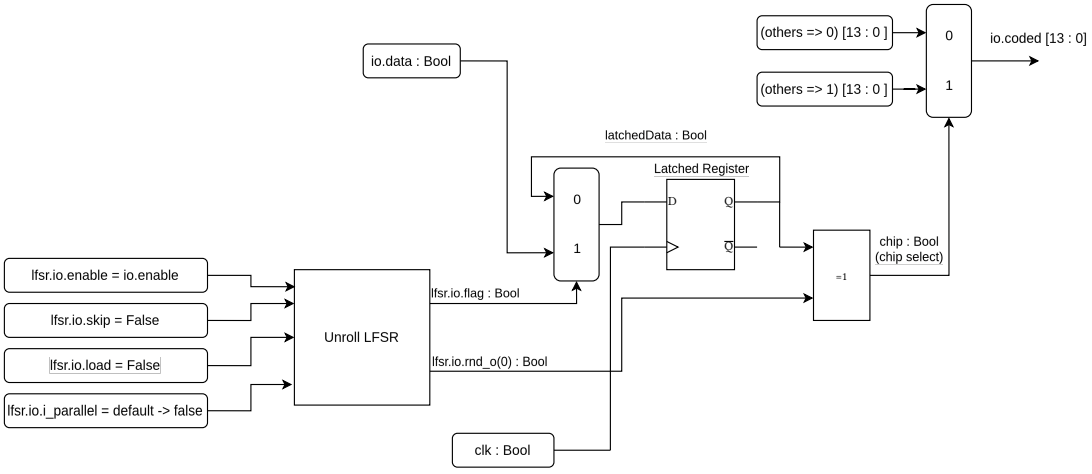

Fig. 3. Transmitter block diagramm

N chips relative to correlator 0, so that together the N correlators cover the entire code period. Each correlator multiplies its delayed code replica with the sampled signal and integrates the result over one code period, marked as "Integrate & Dump" in Fig. 5. At the end of every code period, this new correlation value is compared against the previous maximum. Should it be higher, "New Maximum?" is asserted and the code detected signal is pulled to high. The local code replica is then advanced by one additional chip before the next code period begins, so that over time each correlator sweeps across its assigned slice of the code phase. Once every possible offset has been tested, the system evaluates the Code_Acquisition blocks. The acquisition block that found the highest correlation peak (maxReg) wins. Its internal LFSR state is put on the i_parallel input of the main tracking LFSR, and a load command is issued, to load it in and use it.
This overwrites the main LFSRs state with the LFSR state where the highest correlation was found. From there on, the main LFSR continues shifting in its usual manner to stay synchronized with the transmitter. In this projects implementation, the Receiver_Analog module instantiates an array of 32 of these parallel Code_Acquisition blocks to perform this search. b) *Tracking and Demodulation:* Once synchronized, the receiver must constantly adjust its code phase to account for clock drift. This is implemented by using three parallel correlators: Early, Prompt, and Late.
The receiver takes the incoming ADC signal and passes it through a short shift register (inputRegVec), resulting in three different versions of the signal. The current immediate signal (Early), delayed by one cycle (Prompt), and delayed by two cycles (Late). These three signals are correlated simultaneously against the single output of the main tracking LFSR. At the end of each symbol period, one data bit, the correlation results are passed into a Delay-Locked Loop (DLL). The DLL compares the absolute values of the three accumulators. If the Early or Late correlator is consistently stronger than the Prompt correlator, the DLL increments or decrements an internal 16-bit error register.
To prevent the system from correcting too quickly due to random noise, the DLL requires this error register to be higher than a given threshold before adjusting itself.
Should the threshold exceeded positively, the DLL issues an advance command. The receiver handles this by disabling the tracking LFSR for one cycle. This pauses it to let the incoming signal catch up. If instead the threshold is exceeded negatively, it gives a delay command, sending the skip signal to the LFSR to jump an extra step forward.
The actual data demodulation is read from the Prompt accumulator. If the accumulated value is greater than zero, a '1' is output and otherwise, a '0' is output.
The whole setup of the receiver can be seen in the Fig. 19, which shows all the parts described in this section as one block diagram.

F. Channel Emulation

The channel is the middle part between the transmitter and receiver. Its is used to simulate a real-world environment by adding noise to the output of the transmitter, before the signal reaches the receiver.
The channel applies two main effects onto the signal of the transmitter, the attenuation and noise. Attenuation is implemented using an arithmetic right-shift on the 14-bit signal. This gives control over the signals amplitude.
Since the FPGA lacks floating-point units, the decision was made to use a simpler approximation based on the Central Limit Theorem for noise generation instead of proper AWGN.
The channel instantiates five separate unrolled LFSRs, each with different lengths and feedback polynomials. This makes sure that the sequences of the LFSRs are uncorrelated. By continuously adding up the outputs of the five pseudorandom LFSR generators, the resulting noise approximates a Gaussian distribution. This resulting noise value can then be based on the desired noise floor and added to the attenuated signal during runtime. This allows control over noise and attenuation while the FPGAs are running.

G. Top-Level Integration and controlling the FPGAs

The top-level components, tx_top, rx_top, and channel_top, are used to tie the physical pins of the FPGA development boards to the implemented receiver, transmitter and channel designs. The top-level are all still implemented in SpinalHDL, and pins are assigned using the set_name() method of SpinalHDL, which allows is to name the I/O signals to the correct pins.
Another addition at the top level is clock domain handling for the external 14-bit DAC and ADC. To ensure the DAC receives stable data and meets required setup and hold times, the tx_top module implements a falling-edge clock domain. The encoded output chips-bits of the transmitter are passed into this falling-edge buffer right before being driven out to the DAC pins.
The top-levels of the system must also translate between different binary formats. The internal logic uses two's complement mathematics to easily handle positive and negative values. However, the external ADCs and DACs operate using offset binary and to convert between the two, the top-level modules invert the MSB of the output data. For example, in the transmitter, tx.io.coded.asBits(13) flips the sign bit before it is send out to the DAC. The receiver then applies the same inversion to convert the ADCs offset binary back into twos complement for the tracking loops.

H. Known Limitations

To reflect on a few design trade-offs, this section focuses on the known implementations.
Typical receivers rely on a finely adjustable PLL or numerically controlled oscillator (NCO) to match the receiver frequency to the transmitted signal frequency [10]. For this, either control signals emitted by the code acquisition and tracking modules, or a carrier tracking loop are used. The Cyclone II FPGA does not contain such fixed function blocks [12]. While a fully digital implementation is possible on an FPGA without specialized hardware [13], it is considered out of scope for this project. Instead, for frequency tracking,

the receiver reference code generator is periodically skipped forward or delayed by one cycle, whenever the accumulated error signal of the Code Tracking Block exceeds a chosen threshold.

a) *No re-acquisition after loss of lock:* The winning correlation value inside each individual acquisition engine, is only ever allowed to increase. Once the receiver has locked on, there is currently no mechanism to detect a lost faded signal and trigger a new acquisition search. Recovering currently requires an external reset press on the FPGAs button.

b) *Alternatives to Exhaustive Search with Parallel Correlators:* During the design of the acquisition phase, there were ideas to use an FFT based correlation instead of out current approach. However, an FFT-based approad was deemed too resource-intensive for this board and running 32 parallel correlators with the unrolled LFSR was the most practical choice.

I. Implementation on FPGA

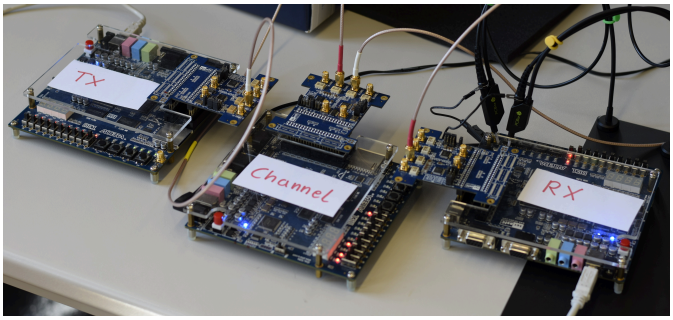

Fig. 6. Full Setup on Cyclone II FPGAs

The full setup on FPGAs is shown in image Fig. 6. The transmitter is marked as TX on the left side of the image, connected to the channel in the middle. On the right, the receiver board then receives the data from the channel. The three FPGA boards are controlled via their onboard switches (SW) and buttons (KEY). For easier understanding, a table overview of the different FPGAs controls is given:

TABLE I
HARDWARE KEY AND SW MAPPING FOR THE TRANSMITTER, CHANNEL, AND RECEIVER FPGA BOARDS.

Board	Interface	Function
All Boards	KEY0	Asynchronous active-low system reset
All Boards	SW(0)	Enable switch
Transmitter (TX)	SW(1)	Data input bit
Channel	SW(5) – SW(2)	Set signal attenuation level
Channel	SW(9) – SW(6)	Set noise level
Channel	LEDR(9) – LEDR(6)	Visual indicator for noise switch state
Receiver (RX)	LEDG(0)	Receiver synchronization status (syncd)
Receiver (RX)	LEDG(1)	Visual indicator for decoded data output bit

This allows for the adjusting the transmitter, receiver and channel while the transmission is running.

IV. Simulations

A. Bit-Error Rate of an Ideal Receiver in the Presence of Noise

A test sequence of random bits is generated and encoded. Then, random gaussian noise is added so that

$$\mathrm{SNR} = \frac{S^2}{N^2} = \frac{1}{\sigma^2} \tag{14}$$

where $S^2 = 1$ is the power of the signal and σ^2 is the power of the noise.

To simulate and evaluate an ideal DLSS receiver with zero frequency error δf and time error $\delta\tau$, the resulting noisy signal is split into code length sized bins, and the correlation of the code with each of the bins is computed. The bit decision is made at a decision threshold of 0.

The result is shown in Fig. 7. Up to a SNR of -20dB, the BER is insignificant.

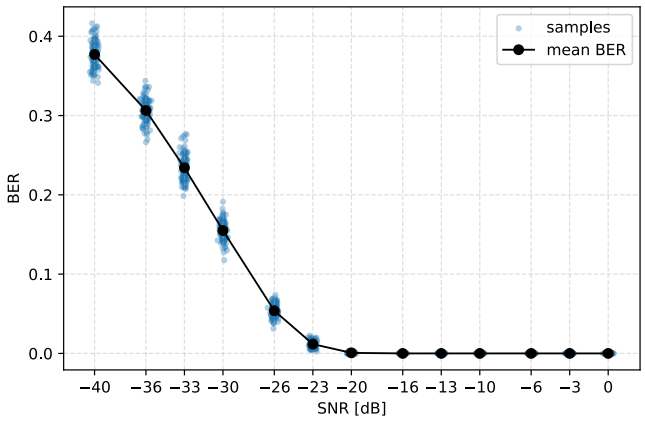

Fig. 7. BER of ideal receiver with decreasing signal-to-noise-ratio, assuming successful code acquisition.

B. Required Integration Period for successful Code Acquisition

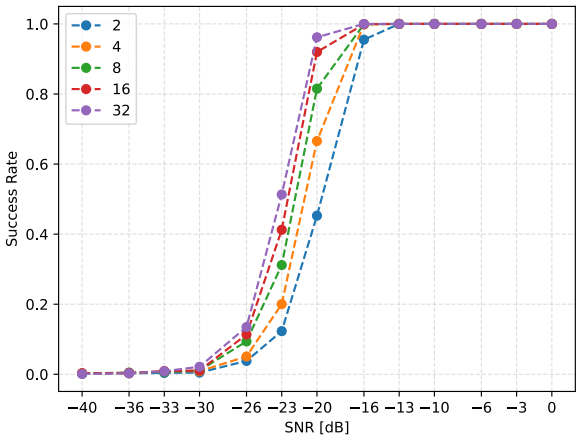

Fig. 8. Code acquisition success rate with decreasing signal-to-noise-ratio, for different integration times T given as multiples of the code length.

The integration time required for code acquisition is in a linear relationship with the SNR[10]:

$$\mathrm{SNR} = T\sqrt{\nu}\frac{C}{N_0} \tag{15}$$

where $\frac{C}{N_0}$ is the Carrier-To-Noise ratio, T is the integration time, and ν is the number of averaged noncoherent integrations. As the SNR decreases by 3dB, the required integration time doubles.

Fig. 8 shows the increasing success rate with larger integration time T as the result of a Monte-Carlo simulation.

Increasing the coherent integration time T comes with the following downside: As T increases, the residual frequency offset δf between transmitter and receiver gets accumulated, eventually leading to a decreased correlator result. To avoid this,

$$T < \frac{T_C - \delta\tau_{\text{initial}}}{T_C} \cdot \delta f \tag{16}$$

should be observed, where T_C is the chip period and $\delta\tau_{\text{initial}}$ is the time offset at the beginning of the correlator integration period, with $\delta\tau_{\text{initial}} \ll T_C$ for a correlator that successfully "finds" the transmitted signal.

C. Pseudo-AWGN generation

A single LFSR produces a quasi-sinc shaped line spectrum [11]. To emulate an additive white gaussian noise channel, approximately gaussian noise can be generated using the central limit theorem, by summing the outputs of multiple LFSRs [3].

To optimize the choice of parameters quickly, a simulation was performed in python.

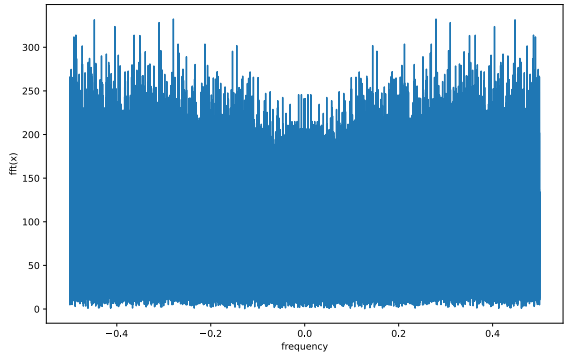

Fig. 9. Fourier transform of 1×10^5 generated noise samples using the first configuration of LFSRs

An initial estimate was made that five LFSRs of 30 to 32 bit length would produce noise which is sufficiently uncorrelated to the used code, produced by a 10 bit LFSR.

A 12-bit output is taken from each of the LFSRs and summed using a saturating addition function, preventing overflow of the 14-bit output value.

Each of the shift registers uses a different polynomial, and also is shifted a different number of steps each cycle.

Fig. 9 shows the initial configuration with relatively short LFSR step lengths per cycle. Clearly visible is relatively low power at lower frequencies. By experimentally adjusting the parameters, the much more "white" spectrum shown in Fig. 10 was achieved.

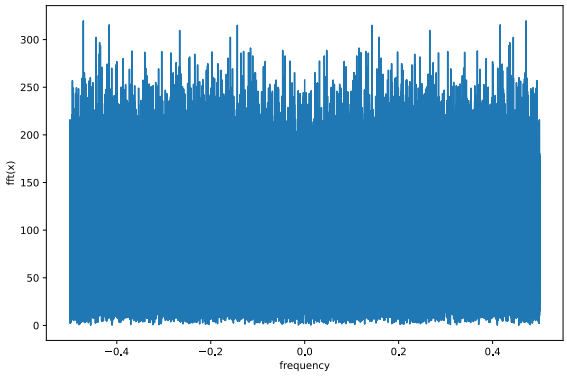

Fig. 10. Fourier transform of 1×10^5 generated noise samples using the optimized configuration.

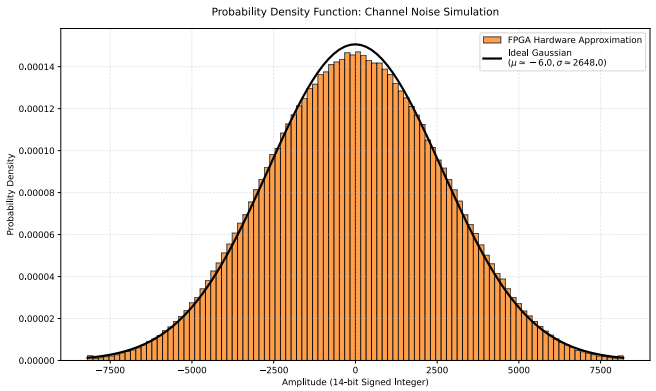

Fig. 11. Approximated PDF of generated noise. The extreme bins, containing the maximum and minimum of the 14-bit signed integer value, respectively, show a slightly inflated ocurrence due to the applied saturating addition function.

Alternatively, the Box-Muller method can be used [3], however this requires the use of more complex mathematical operations, which would have to be specially implemented on the FPGA.

D. Impact of Quantization and Truncation

Initial simulations in python were performed using floating-point numbers. To verify the applicability of simulation results to the hardware implementation, the generated noise is scaled and clipped to fit a 14-bit integer or fixed point value.

The level of the signal is scaled to [−64,+64] to allow for dynamic range to fit a SNR of up to −40dB. Clipping values outside the range $[-2^{13}, 2^{13}-1]$ does not change the simulation results, confirming the validity of the approach.

V. Physical Verification

To verify the function of each module, first, the transmitter is directly connected to an oscilloscope. The capture is shown in Fig. 12. Clearly visible are very sharp signal edges, showing spectral content at high frequencies.

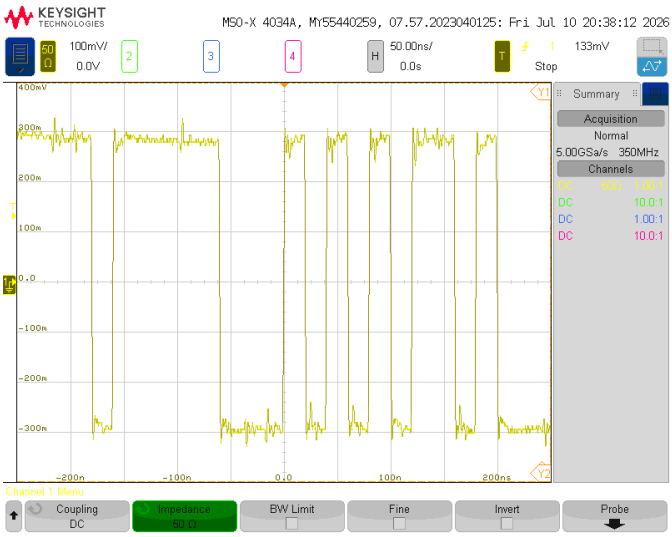

Fig. 12. Oscilloscope capture of the transmitter output signal.

Next, a 11MHz low-pass filter (LPF) is connected between the transmitter and the oscilloscope. The capture in Fig. 13 shows that the signal cannot reach the full level at every chip transition anymore.

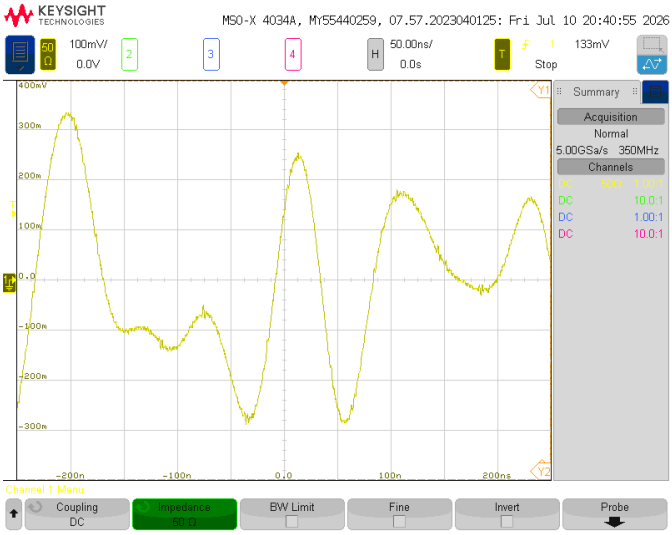

Fig. 13. Oscilloscope capture of the 11MHz low-pass filtered transmitter output signal.

Fig. 14 shows the transmit signal at a wider time span, as well as a spectrum of the trace computed by the oscilloscope. The shape of the LPF can be seen as the outline of the spectrum, while the transmit signal generates a comb-like spectrum.

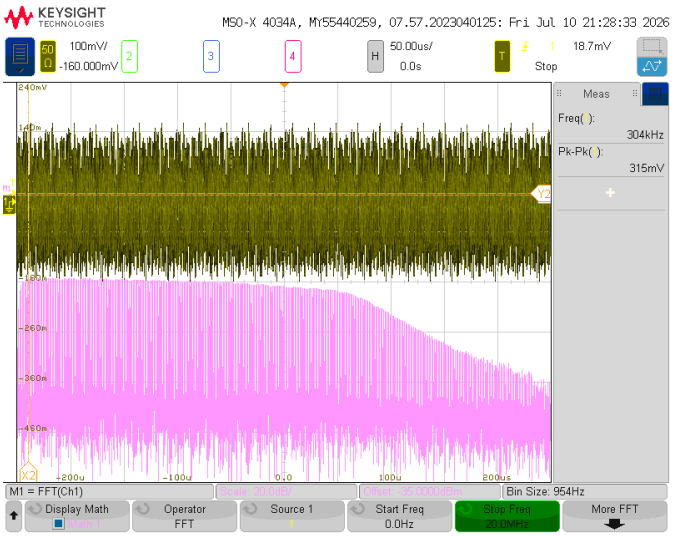

Fig. 14. FFT of the 11MHz low-pass filtered signal, showing the filter characteristic as well as the comb-like structure of the transmit signal, as well as the repetitive nature of the code.

To investigate the comb structure in more detail, a zoomed-in spectrum is shown in Fig. 15. The distance between the peaks is measured at roughly 48 kHz, matching the code repetition rate of $50\frac{\text{MHz}}{1023\ \text{chips}} \approx 49$ kHz.

Fig. 15. FFT of the 11MHz low-pass filtered signal, showing the 48 kHz spacing of the comb-like transmit signal.

To verify the function of the receiver, the transmitter and receiver are first connected directly, without channel emulator.
The data input of the transmitter is fixed to either “1” or “0”, so that the data output of the receiver outputs a pulse whenever a bit error occurs, and otherwise stays at a constant value.

When the receiver fails to track the code or repeatedly loses the lock, it outputs a rapid series of pulses as shown in Fig. 16.

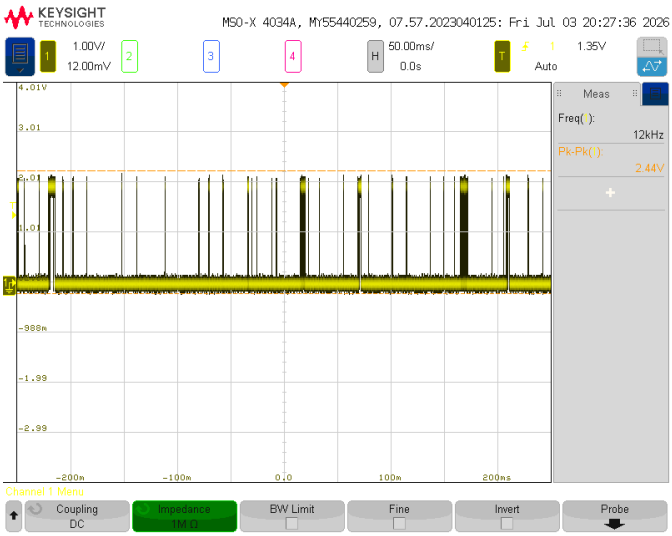

Fig. 16. Capture of the receiver data output in a lost-lock state.

Once we tuned the control loops of the receiver to a degree that they were able to reliably lock onto and track a noise-free signal, the channel emulator was inserted between transmitter and receiver. Basic function could be verified, however, a detailed bit-error-rate analysis was not performed, as the performance of the implemented code tracking loop was not sufficient and required more tuning. Once the receiver fully loses lock to the signal, the BER is 50%, at significantly higher SNR levels than shown in Fig. 7.

Fig. 17 shows the code acquisition process. As no threshold to determine whether a signal has been found has been implemented, the receiver outputs data as soon as it is enabled. Whenever a new maximum correlation offset is found, the receiver shifts to this offset. Thus, until the actual signal is found, the value of the output is random. Once the signal has been acquired, the output settles to the correct value.

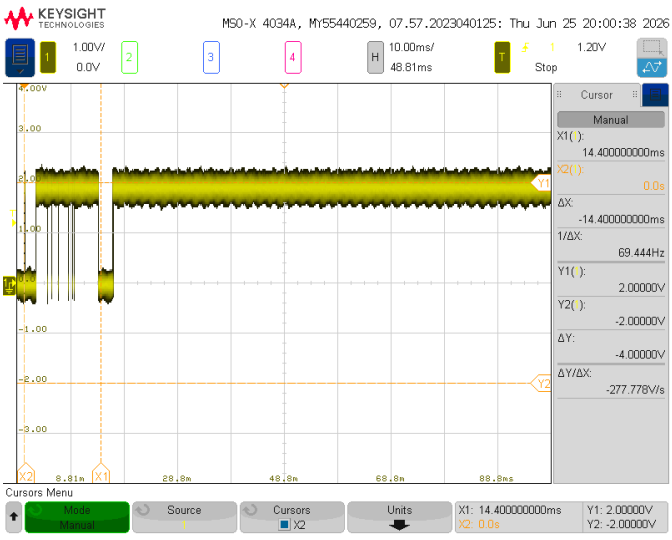

Fig. 17. Capture of the receiver data output during initial code acquisition. The transmitted data bit is fixed to “1”. During code acquisition, the receiver attempts to decode data from every candidate code offset as they are found, so random data is output until finally the correct code offset is acquired and locked onto.

Fig. 18 shows a more detailed view of this process. The time to acquire the signal using a single correlator is measured at 16ms, thus the receiver computed roughly 750 correlations until the signal was acquired. To reduce the acquisition time, additional correlators were implemented. 32 Correlators cover the entire code space in 32 code periods, the expected time to lock is reduced to less than 1ms. Doubling the number of correlators again to 64, to halve the time to lock once more, exceeds the available hardware on the used FPGA.

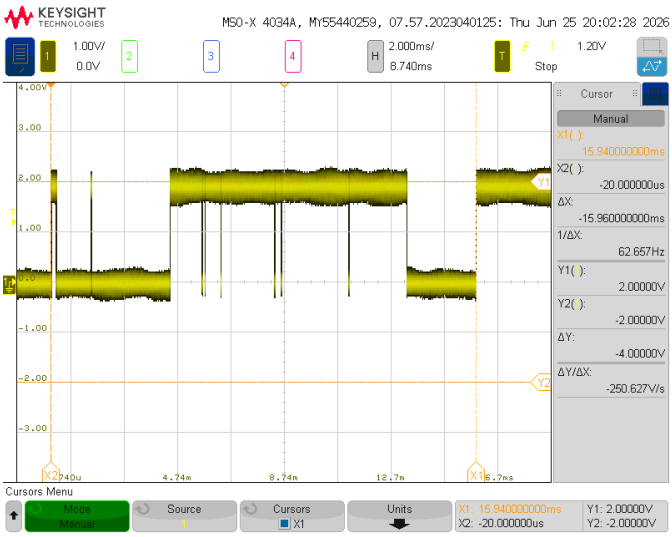

Fig. 18. Shorter time base capture of the receiver data output during initial code acquisition. The acquisition process took 16ms.

VI. Outlook

The system developed in this project could be extended to Several parts of this design were built with a working system as the first priority, not resource efficiency or performance. This chapter collects a few ideas and directions for further optimization that based on the choices made in the current implementation, several of which map onto standard GNSS receiver design trade-offs described in the literature [10].

A. ADC Sampling Rate

In our implementation, the signal is sampled at the chip rate. Sampling at integer multiples of the chip rate can introduce distortions that significantlyreduce tracking performance [8]. To reduce this effect, jitter can be introduced to the correlator, with a relatively small amount of additional hardware resources [8].

B. ADC bit depth

It is possible to reduce the number of quantization levels of the analog-to-digital converter, down to as little as one or two bits, without a significant loss in correlation performance, while drastically reducing the hardware complexity of the receiver [6].
In our implementation, the ADC bit depth n_adc is used as a parameter through the entire signal path rather than hardcoded. All 32 parallel Code_Acquisition correlators, the three tracking correlators (Early, Prompt, Late), and the DLL's input ports sizes are defined as n_adc + m_lfsr bits wide. Reducing n_adc does not only shrink the ADC itself, it shrinks every accumulator and adder in all 35 correlators at once.
For the acquisition stage, 32 parallel correlators were chosen as a tradeoff between search time and FPGA logic usage, the logic freed up by a smaller n_adc could be reinvested into more parallel search engines instead. This would improve acquisition time without a larger FPGA.

C. DLL loop filter design

The current DLL increments or decrements errorReg by a fixed step (+|2/-1) each cycle the early or late accumulator dominates. Then, an advance or delay command is set when errorReg crosses a hardcoded threshold of 20.
Comparing this to a standard approach in GNSS receiver design specifies a DLL loop filter directly in terms of a target noise bandwidth B_n, with the filter order and B_n together determining the trade-off between residual tracking-noise jitter and how quickly the loop can respond to genuine code-phase drift [10]. Since code tracking loops are usually implemented with a first-order filter, replacing the current threshold based implementation with a filter parameterized directly by a chosen B_n would make this an adjustable design parameter rather than somewhat empirically-chosen constants. This would also make the loop's expected steady-state tracking error predictable rather than something to be found only through simulation.

D. Correlator spacing

The code tracking correlators are spaced one chip apart. Prerequisites have been made to easily allow reducing the chip rate to 25 MHz while keeping the clock speed of 50 MHz, thus reducing the correlator spacing to half-chip. Additionally, more complex code tracking implementations with more correlators can be studied.

E. Channel emulator

The channel emulator can be extended to emulate multipath channels in addition to AWGN channels. For this, shift registers that produce delayed copies of the signal at the input could be used. Additionally, the spectrum of the channel can be shaped by implementing various digital filters.

VII. Conclusion

This project successfully designed and implemented a complete DSSS communication system in hardware. By splitting the project into three parts, across three DE1 FPGA boards, the project covered a working transmitter, an adjustable noise generator, and a receiver that handles both code acquisition and tracking.

A. Direct-Sequence Spread Spectrum Conclusion

The implementation of the DSSS system proved that the theoretical concepts translate well into a working hardware

design. Despite the limitations of a strict 50 MHz clock and no floating-point math, the main logic worked as expected and the receiver managed to correctly receive data from the transmitter. Even when connection the channel between transmitter and receiver, the receiver was still able to find the correct code phase. Once locked, the Delay-Locked Loop was able to successfully track the signal and recover the original data bits. In conclusion, while there is room for future optimization, such as implementing a formal loop filter or automatic re-acquisition, the DSSS system still works in its current state.

B. SpinalHDL Conclusion

Using SpinalHDL as the hardware description language proved to be a advantage for this project. It made implementing the designs, like the 32 parallel correlators and the unrolled LFSRs, easier than writing standard VHDL or Verilog by hand. This was also due to the benefit that SpinalHDL was deemed as more accessible to beginners. Some of the group members were relatively new to hardware design and had limited experience with VHDL or Verilog. Still, because SpinalHDL offers high-level software abstractions, the design process was manageable for them and the ability to use standard software paradigms saved a significant amount of development time.

In summary, the project met its main goals. The final hardware successfully transmits and recovers data. This proves that the theoretical DSSS concepts and the SpinalHDL design approach work well.

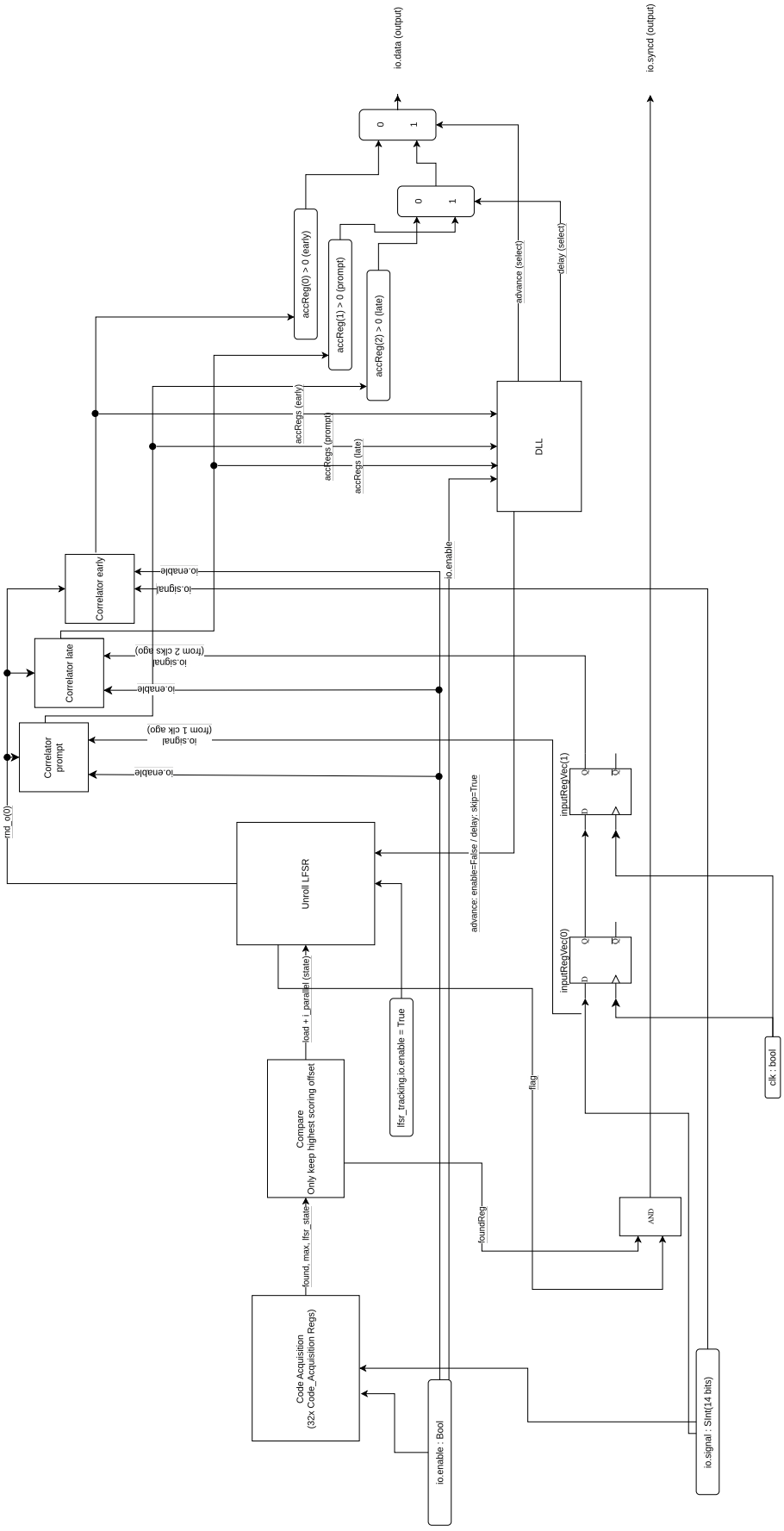

Fig. 19. Receiver block diagramm

References

[1] R. L. Pickholtz, D. L. Schilling, and L. B. Milstein, "Theory of Spread-Spectrum Communications—A Tutorial," *IEEE Transactions on Communications*, vol. 30, no. 5, pp. 855–884, 1982.

[2] E. Kaplan and C. Hegarty, 2017.

[3] P. Michel C. Jeruchim and K. S. S. Balaban, *Simulation of Communication Systems*. 2006, pp. 383–384. doi: https://doi.org/10.1007/b117713.

[4] C. Shannon, "Communication in the Presence of Noise," *Proceedings of the IRE*, vol. 37, no. 1, pp. 10–21, 1949, doi: 10.1109/JRPROC.1949.232969.

[5] B. V. V. Satyanarayana, N. S. S. Teja, M. Pavan, K. Smrini, G. P. Kumar, and P. R. Budumuru, "FPGA Implementation of Correlator for Direct Sequence Spread Spectrum (DSSS) Systems," in *2024 International Conference on Integrated Circuits and Communication Systems (ICICACS)*, 2024, pp. 1–8. doi: 10.1109/ICICACS60521.2024.10498616.

[6] C. J. HEGARTY, "Analytical Model for GNSS Receiver Implementation Losses," *NAVIGATION*, vol. 58, no. 1, pp. 29–44, 2011, doi: https://doi.org/10.1002/j.2161-4296.2011.tb01790.x.

[7] V. T. Tran, N. C. Shivaramaiah, T. D. Nguyen, J. W. Cheong, E. P. Glennon, and A. G. Dempster, "Generalised Theory on the Effects of Sampling Frequency on GNSS Code Tracking," *Journal of Navigation*, vol. 71, no. 2, pp. 257–280, 2018, doi: 10.1017/S0373463317000741.

[8] V. T. Tran, N. C. Shivaramaiah, T. D. Nguyen, E. P. Glennon, and A. G. Dempster, "GNSS receiver implementations to mitigate the effects of commensurate sampling frequencies on DLL code tracking," *GPS Solutions*, vol. 22, no. 1, p. 24, Dec. 2017, doi: 10.1007/s10291-017-0690-x.

[9] A. Wilde, "The Generalized Delay-Locked Loop," *Wireless Personal Communications*, pp. 113–130, 1998, doi: 10.1023/A:1008851125419.

[10] T. Pany and J.-H. Won, "Springer Handbook of Global Navigation Satellite Systems - Signal Processing," 2017, pp. 401–440. doi: 10.1007/978-3-319-42928-1_14.

[11] S. Orth and H. Klingbeil, "Maximum length binary sequences and spectral power distribution of periodic signals," *EURASIP Journal on Advances in Signal Processing*, vol. 2024, no. 1, p. 80, Jul. 2024, doi: 10.1186/s13634-024-01177-5.

[12] "Cyclone II Device Handbook, Volume 1." [Online]. Available: https://docs.altera.com/v/u/docs/654376/cyclone-ii-device-handbook

[13] W. Deng *et al.*, "A Fully Synthesizable All-Digital PLL With Interpolative Phase Coupled Oscillator, Current-Output DAC, and Fine-Resolution Digital Varactor Using Gated Edge Injection Technique," *IEEE Journal of Solid-State Circuits*, vol. 50, no. 1, pp. 68–80, 2015, doi: 10.1109/JSSC.2014.2348311.